



Implementación de Reinforcement Learning en SOAR para Recorrido de Laberintos

Marcelo Ferreiro Sánchez
José Romero Conde

Resumen

En esta memoria se documenta el desarrollo e implementación de un experimento de navegación de laberintos utilizando las técnicas y algoritmos de aprendizaje por refuerzo que la arquitectura cognitiva SOAR provee de forma nativa. Se mostrarán experimentos y resultados en varios laberintos genéricos, generados proceduralmente y posteriormente en uno específico, donde se usará la arquitectura para solucionar un problema de recreación histórica.

Índice

1. Introducción	2
1.1. Motivación	2
1.2. Objetivos	2
1.2.1. Objetivo General	2
1.2.2. Objetivos Específicos	2
2. Marco Teórico	2
2.1. Reinforcement Learning	2
2.1.1. Conceptos Fundamentales	2
2.1.2. Algoritmo SARSA	3
2.2. Arquitectura Cognitiva SOAR	3
2.2.1. Características Principales	3
2.2.2. Reinforcement Learning en SOAR	4
2.2.3. Flujo de Recompensas	4
2.3. Navegación del laberinto	5
2.3.1. Definición del problema	5
2.3.2. Sensores	5
3. Metodología	5
3.1. Diseño del Sistema	5
3.1.1. Arquitectura General	5
3.1.2. Representación del Laberinto	5
3.1.3. Representación del Estado	6
3.2. Implementación en SOAR	6
3.2.1. Reglas de Producción	6
3.2.2. Función de Recompensa	7
3.2.3. Parámetros de Aprendizaje	8
3.2.4. Métricas de Evaluación	8
4. Experimentos y Resultados	8
4.1. Configuración Experimental Inicial	8
4.2. Resultados	8
4.2.1. Resultados en Laberinto 8×8	9
4.2.2. Resultados en Laberinto 16×16	11
4.2.3. Resultados en Laberinto 32×32	13
4.3. Experimentación en caso real	14
4.3.1. Resultados	16

5. Conclusiones y trabajo futuro	19
A. Código Completo SOAR	19
A.1. Reglas de Elaboración	19
A.2. Reglas de Propuesta	20
A.3. Templates de RL	21
A.4. Reglas de Aplicación	22
B. Configuración JSON	22
B.1. Ejemplo de Estadísticas Guardadas	22

1. Introducción

La navegación autónoma en entornos desconocidos representa uno de los problemas fundamentales en robótica e inteligencia artificial. Este trabajo explora la integración de técnicas de Reinforcement Learning (RL) con la arquitectura cognitiva SOAR para desarrollar un agente capaz de aprender a navegar en laberintos de manera autónoma.

1.1. Motivación

Las arquitecturas cognitivas como SOAR proporcionan un marco estructurado para el desarrollo de agentes inteligentes, mientras que el Reinforcement Learning ofrece mecanismos de aprendizaje basados en experiencia. La combinación de ambos enfoques permite crear sistemas que no solo razonan simbólicamente sino que también mejoran su comportamiento a través de la interacción con el entorno.

Se consideró esta problemática debido a que la arquitectura SOAR facilita la transferencia sim-to-real (simulación a realidad): las políticas de navegación aprendidas en el laberinto virtual pueden ser posteriormente desplegadas en plataformas robóticas físicas con adaptaciones mínimas.

1.2. Objetivos

1.2.1. Objetivo General

El objetivo es simple: Desarrollar e implementar un agente basado en SOAR capaz de aprender a navegar en laberintos mediante técnicas de Reinforcement Learning.

1.2.2. Objetivos Específicos

- Diseñar una representación del estado del agente y del entorno en la memoria de trabajo de SOAR
- Implementar reglas de producción para la percepción sensorial y la toma de decisiones
- Integrar el mecanismo de RL de SOAR utilizando el algoritmo SARSA
- Diseñar una función de recompensa efectiva
- Evaluar la performance del agente
- Analizar la convergencia del aprendizaje y las estrategias que surgen

2. Marco Teórico

2.1. Reinforcement Learning

2.1.1. Conceptos Fundamentales

El Reinforcement Learning es un paradigma de aprendizaje automático donde un agente aprende a tomar decisiones mediante interacción con un entorno. Los elementos

clave son:

- **Agente:** Entidad que toma decisiones y ejecuta acciones
- **Entorno:** Sistema con el que interactúa el agente
- **Estado (s):** Representación de la situación actual
- **Acción (a):** Decisión ejecutada por el agente
- **Recompensa (r):** Señal numérica que evalúa la acción
- **Política (π):** Mapeo de estados a acciones

El objetivo del agente es aprender una política óptima π^* que maximice la recompensa acumulada esperada.

2.1.2. Algoritmo SARSA

SARSA (State-Action-Reward-State-Action) es un algoritmo de RL on-policy que actualiza la función Q mediante:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)] \quad (1)$$

donde:

- $Q(s, a)$: Valor estimado de tomar la acción a en el estado s
- α : Tasa de aprendizaje (learning rate)
- r : Recompensa recibida
- γ : Factor de descuento
- $Q(s', a')$: Valor de la siguiente acción seleccionada en el nuevo estado

SARSA es on-policy porque actualiza Q-valores basándose en la política que está siguiendo actualmente (incluyendo exploración). Si fuera off-policy, usaría la acción óptima en lugar de la acción tomada, como en Q-learning. SOAR también incorpora Q-learning nativamente y se activa/desactiva mediante configuración del comando `rl -set learning on` [1].

2.2. Arquitectura Cognitiva SOAR

2.2.1. Características Principales

SOAR (State, Operator And Result) es una arquitectura cognitiva de propósito general. Sus componentes principales son:

- **Memoria de Trabajo:** Contiene el estado actual del agente
- **Memoria Procedural:** Reglas de producción (knowledge base)

- **Ciclo de Decisión:** Proceso iterativo de reconocimiento-acción
- **Input-Link:** Interfaz de percepción del entorno
- **Output-Link:** Interfaz de ejecución de acciones

El ciclo de decisión de SOAR consta de las siguientes fases:

1. **Elaboración:** Aplicación de reglas que infieren nueva información
2. **Propuesta:** Generación de operadores candidatos
3. **Decisión:** Selección de un operador para ejecutar
4. **Aplicación:** Ejecución del operador seleccionado

2.2.2. Reinforcement Learning en SOAR

SOAR incluye soporte nativo para RL mediante [1]:

- **RL Templates:** Reglas especiales que crean preferencias numéricas
- **Reward-Link:** Estructura donde se colocan las recompensas
- **Q-Values:** Almacenados como preferencias numeric-indifferent
- **Actualización Automática:** SOAR actualiza Q-valores según el algoritmo configurado

Las RL templates tienen la forma:

```

1 sp {rl*template*action
2   (state <s> ^operator <o> +
3     ^some-feature <value>)
4   (<o> ^name action-name)
5 -->
6   (<s> ^operator <o> = 0) # Q-value inicial
7 }
```

SOAR instancia estas templates automáticamente para cada combinación única de características de estado, creando reglas especializadas con Q-valores independientes.

2.2.3. Flujo de Recompensas

El flujo de recompensas en SOAR sigue este patrón [1]:

1. El entorno (Python) calcula la recompensa
2. La recompensa se coloca en `input-link.reward-info.value`
3. Una regla de elaboración copia el valor al `reward-link`
4. El mecanismo de RL lee del `reward-link` y actualiza Q-valores

Esta separación permite que reglas SOAR filtren o modifiquen recompensas antes del aprendizaje.

2.3. Navegación del laberinto

2.3.1. Definición del problema

Se define como:

- **Estados:** Posiciones (x, y) en una cuadrícula de tamaño $W \times H$
- **Acciones:** $A = \{\text{arriba, abajo, izquierda, derecha}\}$
- **Transiciones:** Deterministas (excepto colisiones con paredes)
- **Objetivo:** Alcanzar posición meta (x_g, y_g)

2.3.2. Sensores

El agente percibe el entorno mediante cuatro sensores direccionales:

- Cada sensor indica si hay "pared." o "camino." en esa dirección
- Los sensores permiten exploración local sin conocer el mapa completo
- Esta información limita el espacio de estados observable

3. Metodología

3.1. Diseño del Sistema

3.1.1. Arquitectura General

El sistema implementado consta de tres componentes principales que interactúan en cada ciclo de decisión:

1. **Entorno Python:** Gestiona el laberinto, ejecuta acciones y calcula recompensas
2. **Agente SOAR:** Toma decisiones basándose en percepción y conocimiento aprendido
3. **Interfaz Python-SOAR:** Sincroniza ambos componentes mediante input/output links

3.1.2. Representación del Laberinto

El laberinto se representa como una matriz bidimensional donde:

- 0: Celda transitable
- 1: Pared
- Dimensiones: 7×7 por defecto, pero configurable
- Posición objetivo: $(6, 6)$ por defecto (esquina inferior derecha)
- Posiciones de spawn: Lista configurable de puntos iniciales (en este caso esquina superior izquierda $(0, 0)$)

3.1.3. Representación del Estado

El estado del agente en Python incluye:

```
1 # Variables de posicion
2 self.x                # Posicion X actual
3 self.y                # Posicion Y actual
4 self.steps            # Pasos en episodio actual
5 self.path              # Lista de posiciones visitadas
6 self.discovered        # Set de celdas descubiertas
7
8 # Variables de episodio
9 self.episode           # Numero de episodio
10 self.total_reward      # Recompensa acumulada
11 self.last_reward       # Ultima recompensa recibida
```

El estado en la memoria de trabajo de SOAR se estructura como:

```
1 (I2 ^robot R1)
2   (R1 ^x 3 ^y 5)
3 (I2 ^goal G1)
4   (G1 ^x 6 ^y 6)
5 (I2 ^sensors S1)
6   (S1 ^up wall ^down path
7     ^left path ^right wall)
8 (I2 ^reward-info RI)
9   (RI ^value -0.85)
10 (I2 ^maze M1)
11   (M1 ^width 7 ^height 7)
12 (I2 ^stats ST)
13   (ST ^steps 42 ^episode 15)
```

3.2. Implementación en SOAR

3.2.1. Reglas de Producción

Las reglas principales implementadas son:

Elaboración de Recompensas Copia la recompensa del input-link al reward-link:

```
1 sp {elaborate*reward
2   (state <s> ^io.input-link.reward-info <ri>
3     ^reward-link <rl>)
4   (<ri> ^value <v>)
5 -->
6   (<rl> ^reward <r>)
7   (<r> ^value <v>)
8 }
```


Propuesta de Movimientos Propone movimientos válidos según sensores:

```

1 sp {propose*move*up
2   (state <s> ^io.input-link.sensors <sens>)
3   (<sens> ^up path)
4 -->
5   (<s> ^operator <o> +)
6   (<o> ^name move ^direction up)
7 }

```

Se implementan reglas similares para down, left y right.

Aplicación de Movimientos Crea comandos en el output-link:

```

1 sp {apply*move*create-command
2   (state <s> ^operator <o>
3     ^io.output-link <ol>)
4   (<o> ^name move ^direction <dir>)
5 -->
6   (<ol> ^move <m>)
7   (<m> ^direction <dir>)
8 }

```

Templates de RL Crean preferencias numéricas para cada estado-acción:

```

1 sp {rl*template*move*up
2   (state <s> ^operator <o> +
3     ^io.input-link.robot <r>)
4   (<r> ^x <x> ^y <y>)
5   (<o> ^name move ^direction up)
6 -->
7   (<s> ^operator <o> = 0) # Q-value
8 }

```

3.2.2. Función de Recompensa

La función de recompensa implementada es:

$$r(s, a, s') = \begin{cases} +500 & \text{si } s' = s_{\text{goal}} \\ -1 - 0,1 \cdot d(s', s_{\text{goal}}) & \text{en otro caso} \end{cases} \quad (2)$$

donde $d(s', s_{\text{goal}})$ es la distancia Manhattan al objetivo:

$$d((x, y), (x_g, y_g)) = |x - x_g| + |y - y_g| \quad (3)$$

Esta función:

- Penaliza cada paso (-1) para fomentar rutas cortas
- Añade penalización proporcional a la distancia
- Otorga gran recompensa al alcanzar el objetivo

3.2.3. Parámetros de Aprendizaje

Los parámetros de RL configurados en SOAR son:

Parámetro	Valor	Descripción
learning-rate (α)	0.1	Tasa de actualización de Q-valores
discount-rate (γ)	0.9	Peso de recompensas futuras
epsilon (ϵ)	0.3	Probabilidad de exploración
learning-policy	sarsa	Algoritmo de aprendizaje

Cuadro 1: Parámetros de configuración de RL en SOAR

3.2.4. Métricas de Evaluación

Las métricas utilizadas para evaluar el desempeño son:

- **Tasa de éxito:** $\frac{\text{episodios exitosos}}{\text{episodios totales}}$
- **Pasos promedio:** Media de pasos por episodio exitoso
- **Recompensa total:** Suma de recompensas en episodio
- **Celdas descubiertas:** Porcentaje del mapa explorado
- **Convergencia:** Episodio donde tasa de éxito estabiliza. Se estableció convergencia al alcanzar un 90 % de éxito en 20 episodios consecutivos.

4. Experimentos y Resultados

4.1. Configuración Experimental Inicial

Se realizaron tres pruebas piloto en laberintos generados, de tamaño 8×8 , 16×16 y 32×32 .

Tamaño del laberinto	Máx. Episodios	Máx. Pasos por episodio
8×8	200	20
16×16	500	60
32×32	2000	150

Cuadro 2: Parámetros de entrenamiento según el tamaño del entorno

4.2. Resultados

Los experimentos se evaluaron mediante dos métricas principales para cada tamaño de laberinto:

1. La evolución del **reward promedio** a lo largo del entrenamiento
2. El **mapa de acción más rentable** aprendida en cada celda

En total se muestran seis gráficas: dos por cada entorno (8×8 , 16×16 y 32×32).

4.2.1. Resultados en Laberinto 8×8

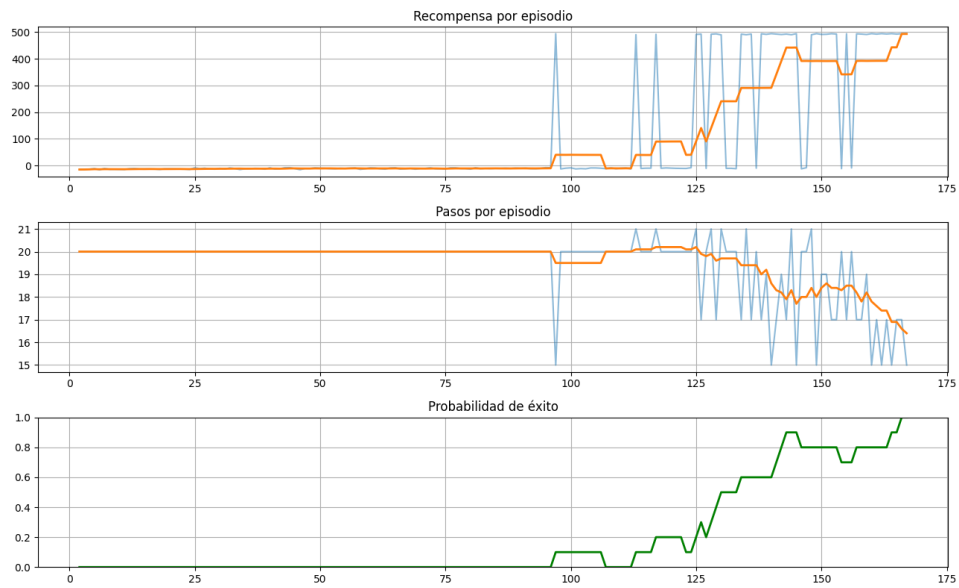


Figura 1: Evolución del reward en el laberinto 8×8 (100 episodios, 20 pasos máx.)

Se aprecia cómo es que el agente aprende, buscando recompensas positivas, también el número de pasos disminuy, lo que indica que la función de recompensa, que penaliza cada paso (en búsqueda de que el robot busque un camino más corto), está cumpliendo su cometido.

También se aprecia cómo es que converge (llega al 90 % de efectividad en los últimos 20 episodios) antes de los 200 episodios establecidos.

4.2.2. Resultados en Laberinto 16×16

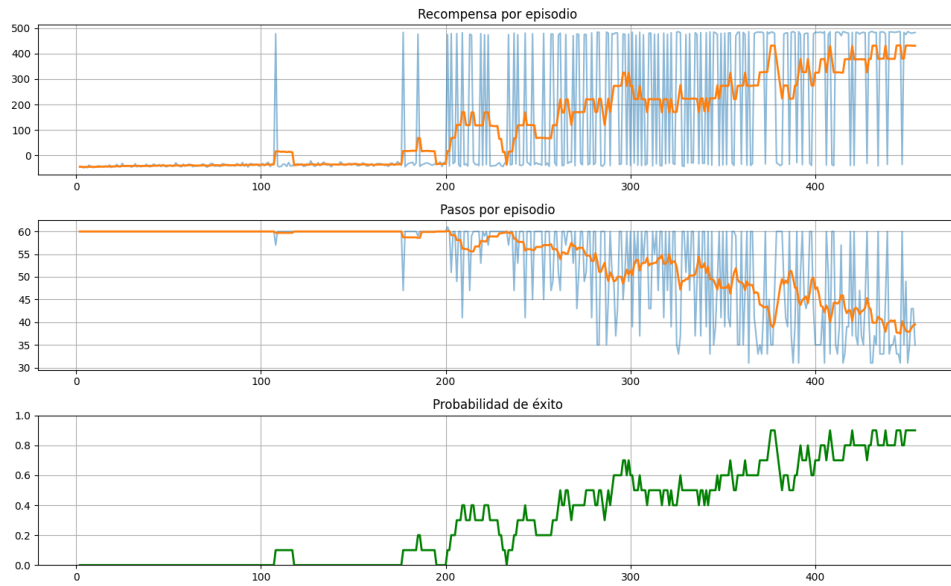


Figura 3: Evolución del reward en el laberinto 16×16 (150 episodios, 60 pasos máx.)

De forma similar, las premisas antes establecidas también se cumplen aquí, el agente es capaz de aumentar la recompensa, converger y disminuir el número de pasos.

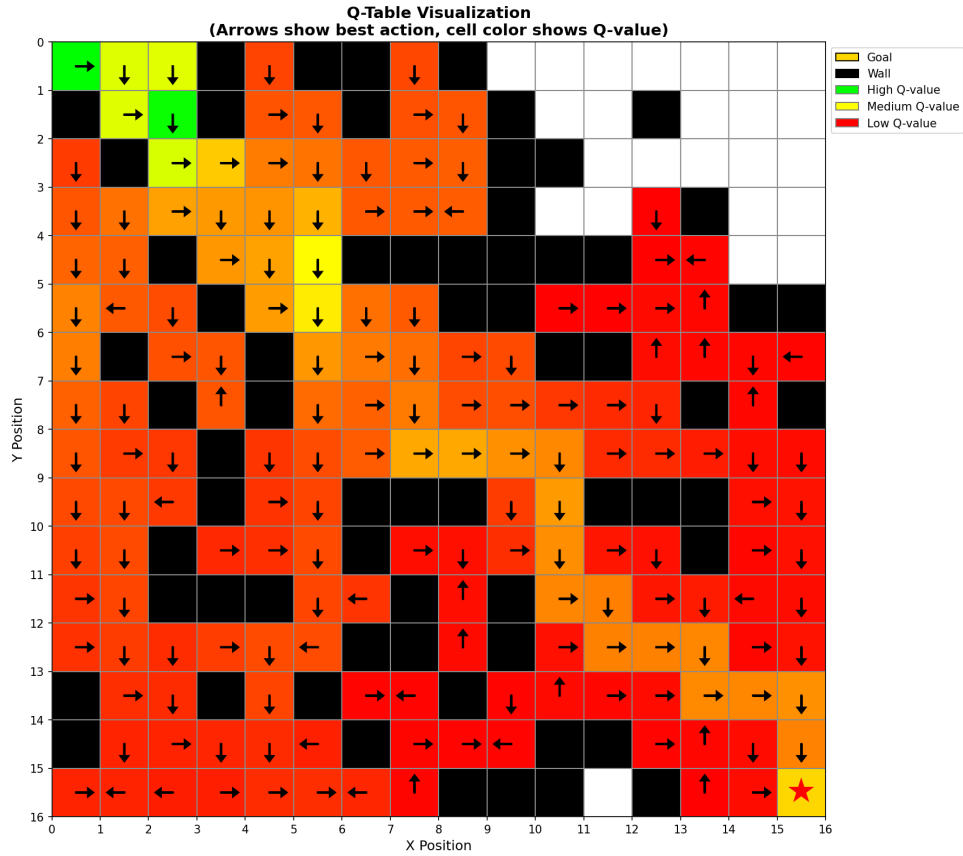


Figura 4: Acción más rentable aprendida en cada celda para el laberinto 16×16

También se observa una política coherente, aunque más compleja debido al mayor tamaño del laberinto, se aprecia también cómo existen celdas no exploradas (en blanco, sin Q-value) ya que el agente no interpretó como rentable el aproximarse en esa dirección.

4.2.3. Resultados en Laberinto 32×32

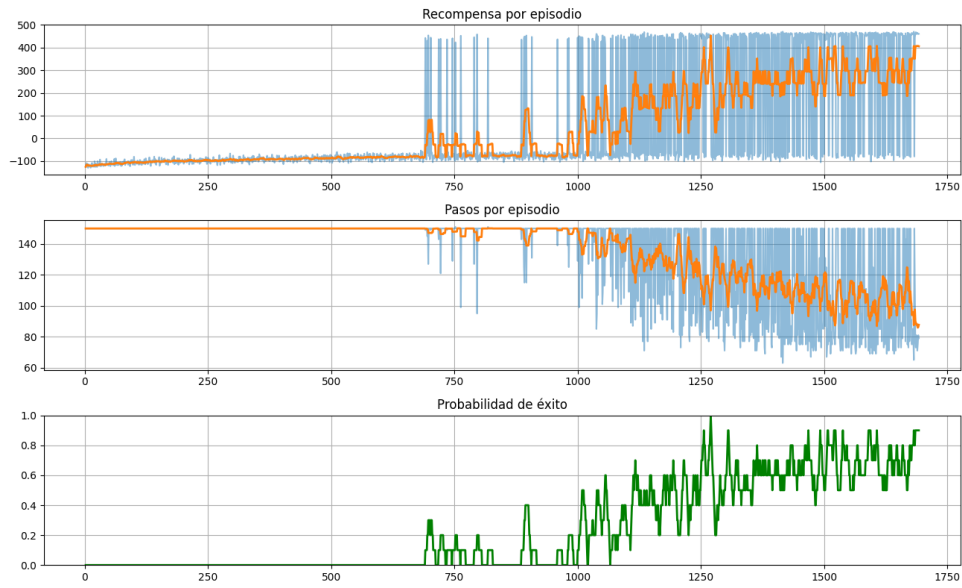


Figura 5: Evolución del reward en el laberinto 32×32 (250 episodios, 150 pasos máx.)

Se cumple lo mismo que en los casos anteriores, aunque la fase de exploración es más prolongada debido al tamaño del laberinto, converge y consigue alcanzar la meta en unos 80 pasos temporales aproximadamente.

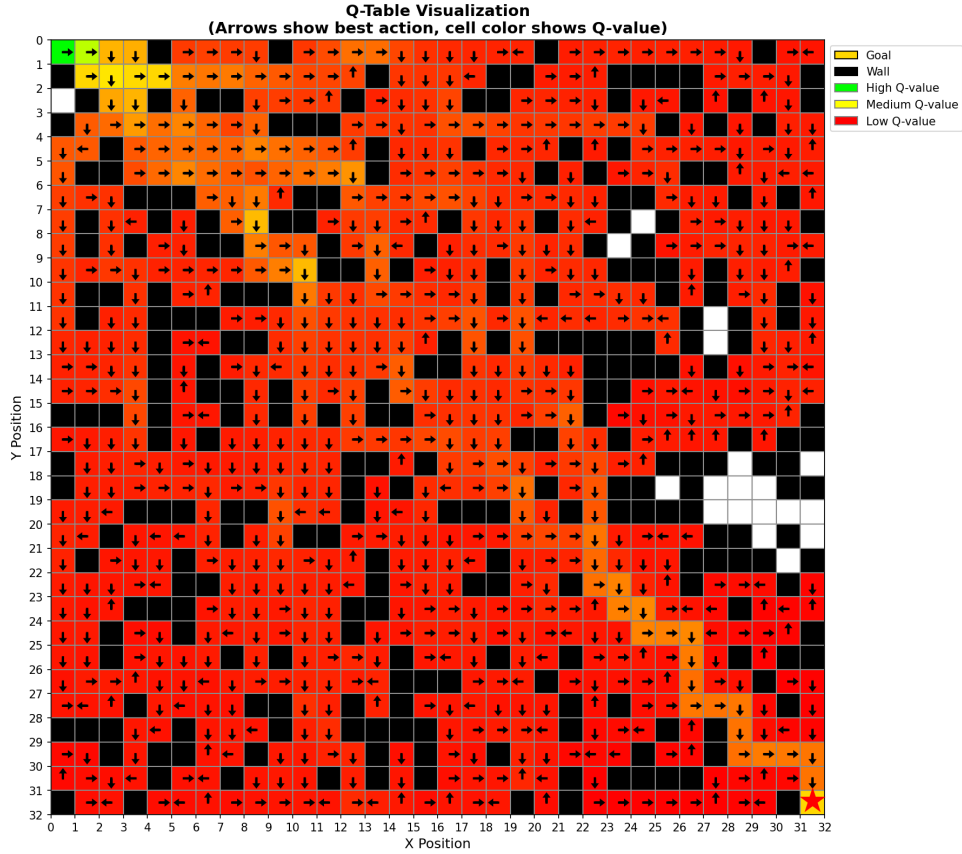


Figura 6: Acción más rentable aprendida en cada celda para el laberinto 32×32

De nuevo, si bien cada vez más difuminado debido al mayor número de celdas y más episodios, se nota claramente un camino coherente hacia el objetivo.

4.3. Experimentación en caso real

Existe la remota posibilidad de que Abraham y Ea-Nasir fueran cohabitantes o incluso vecinos de la ciudad de Ur en la Mesopotamia antigua [3]. Teniendo en cuenta este escenario, y teniendo acceso a los planos de la propia casa del vendedor de cobre [2], es tarea casi obligada el imaginarse y buscar reconstruir el posible camino que uno de sus clientes, en este caso Abraham, podría haber seguido en busca de comprar mercancía de su paisano. Esto es una tarea de la llamada Arqueología Digital y gracias a sistemas de IA como el implementado en este proyecto, es factible hacer estas investigaciones.

Imaginando a Abraham como un hombre de Dios, se puede imaginar que su objetivo al entrar en la tienda sería el de comprar el material lo antes posible para poder ejercer con sus labores de ganadería y pastoreo la mayor parte del día. Es ahí donde el *Reinforcement Learning* hace una labor clave, muy probablemente el camino que el Profeta eligiera sería el más corto y eficiente, del mismo modo que el SARSA.

Así, se ha implementado una versión en cuadrícula 50×50 de la casa de Ea-Nasir (Fig. 7), desde la que el agente SOAR (El Profeta Abraham) parte desde la puerta y se

adentra hasta encontrar al comerciante de Ur.

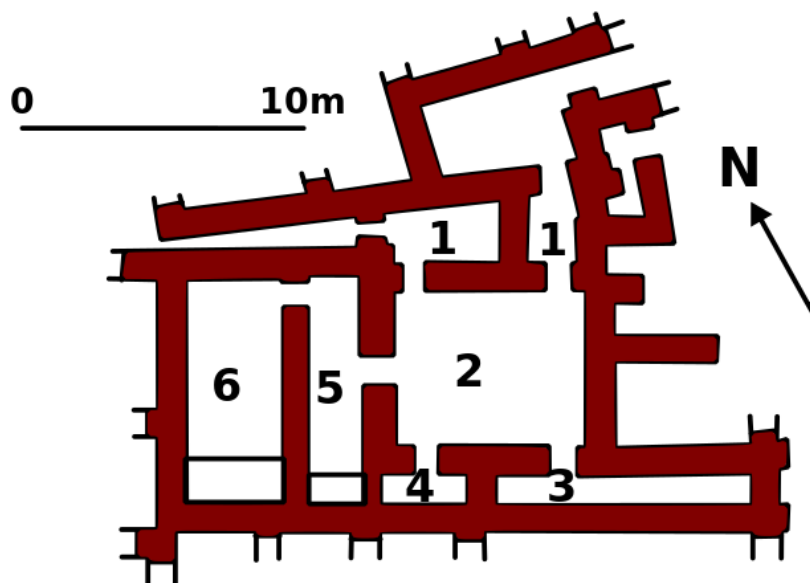


Figura 7: Plano de la casa de Ea-Nasir [2]

A partir de tal representación se puede construir de forma simple (mediante un binarizado con umbral y un reescalado a menor resolución) el siguiente mapa que el agente deberá de navegar.

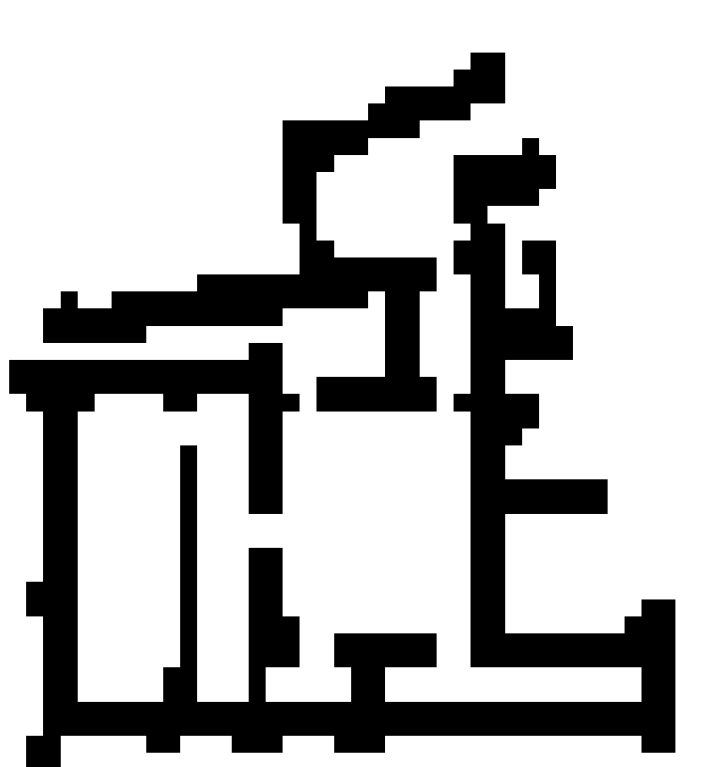


Figura 8: Mapa binario recreado

4.3.1. Resultados

Se encontró que al tratarse de un laberinto de tal tamaño, el agente no era capaz de converger y se estancaba en bucles entre varias celdas relativamente cercanas a la meta (mas con una pared de por medio) consiguiendo así que la recompensa no disminuyera tanto. Incluso cuando el agente conseguía llegar a la meta, la recompensa no era suficiente como para superar el gran número de intentos donde se quedó haciendo bucles cerca del objetivo. Para entender tal dinámica ver el mapa de acciones más rentables aprendidas en la [9].



Figura 9: Mapa de acciones más rentables (intento inicial)

Se aprecia cómo el SOAR estima que lo más rentable es moverse entre esas áreas verdes, aún así existe un leve camino que lleva a la meta pero el agente no opta por él en la mayoría de casos.

Tal dinámica otorgó la oportunidad de implementar una técnica que se descartó por no ser necesaria en los experimentos iniciales (ya que estos son demasiado simples), se trata esta de buscar penalizar las acciones que llevan a celdas ya visitadas. De este modo, cada vez que el agente revisite una celda, la función penalizará con -2 (además de la penalización por paso). Tras aplicar este sencillo cambio, el entrenamiento convergió rápidamente (tendiendo en cuenta que anteriormente se llegó a ejecutar hasta con 10000 sin éxito) en 818 episodios (Fig. 10).

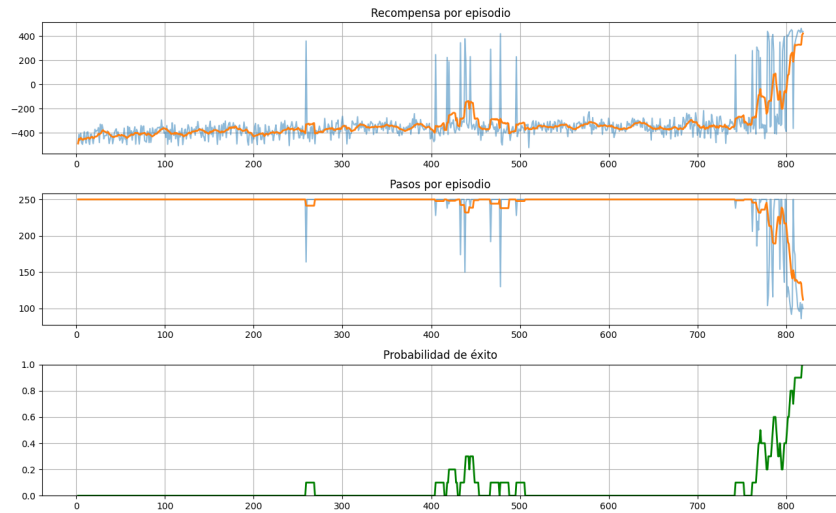


Figura 10: Evolución del reward tras penalizar celdas visitadas

Analizando también el mapa de acciones más rentables aprendidas (Fig. 11), se aprecia cómo el agente ha aprendido una política mucho más coherente y eficiente, evitando bucles y dirigiéndose directamente al objetivo. También cabe notar como es que el agente ha explorado casi la totalidad el mapa, mientras que antes había zonas completamente inexploradas.

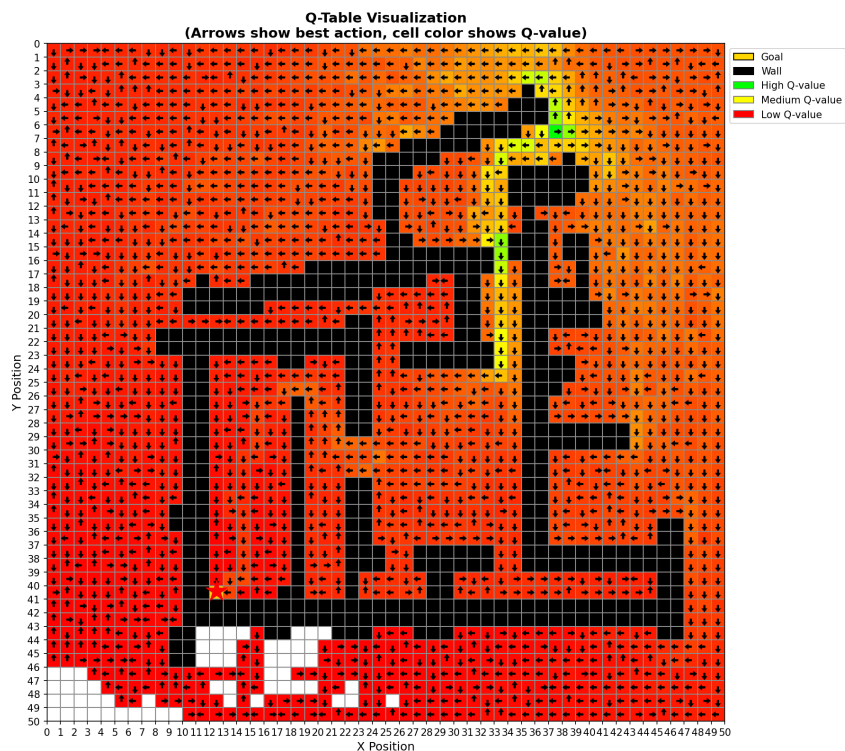


Figura 11: Mapa de acciones más rentables tras penalizar celdas visitadas

Como último experimento se decidió repetir la prueba pero con un *spawn* del agente completamente aleatorio, para comprobar si prodría ser capaz de converger y aprender

una política robusta desde cualquier punto de inicio.

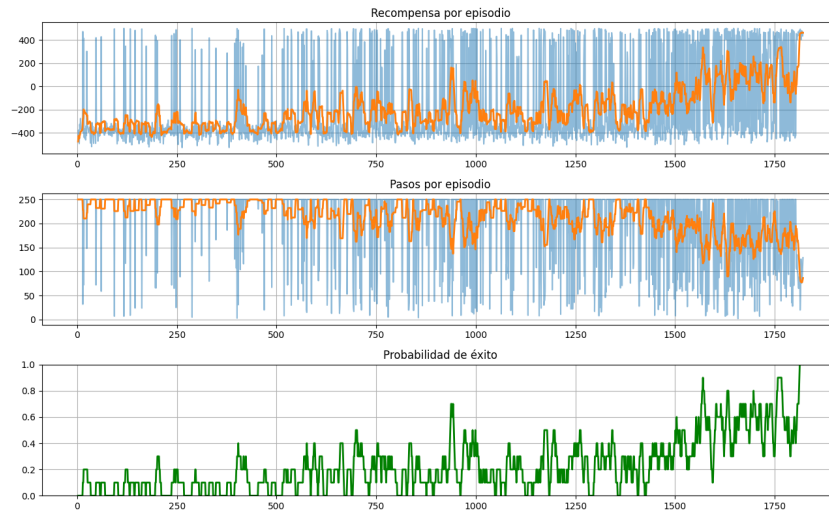


Figura 12: Evolución del reward tras penalizar celdas visitadas

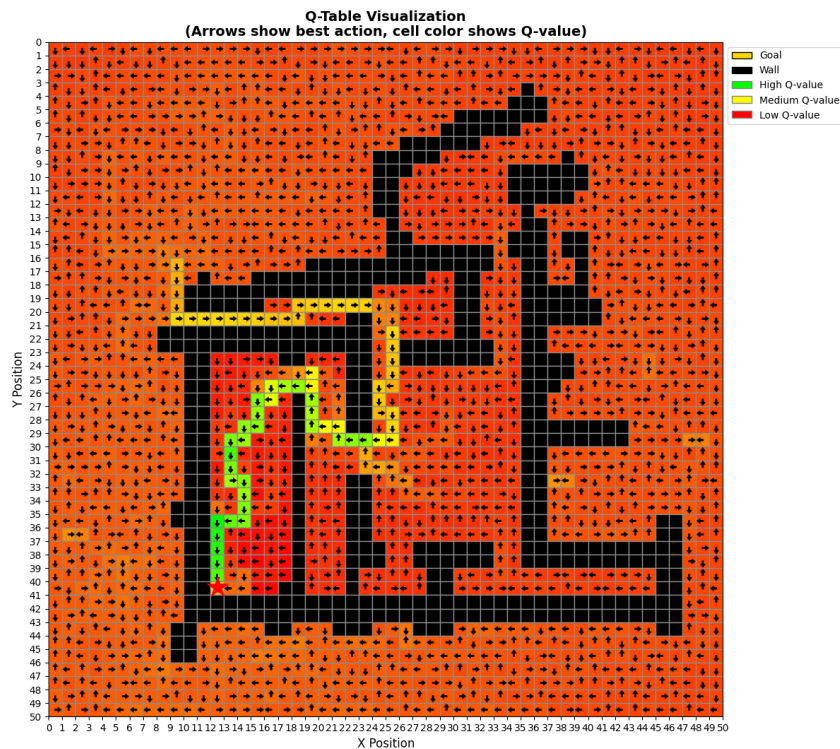


Figura 13: Mapa de acciones más rentables tras penalizar celdas visitadas

Si bien la convergencia fue más lenta (alrededor de 1800 episodios [12]), la política aprendida si vemos el mapa [13] marca mejor el camino más rentable según la qtable,

además de explorar completamente el mapa y garantizar que desde cualquier celda el agente pueda llegar a la meta.

5. Conclusiones y trabajo futuro

Se logró hacer uso de las capacidades nativas que proporciona SOAR para el aprendizaje por refuerzo. La integración entre Python y SOAR permitió gestionar adecuadamente los ciclos de decisión del agente, aunque no sin dificultades debido a que la documentación oficial no es suficientemente completa en algunos aspectos críticos de su funcionamiento.

Los experimentos realizados en laberintos de diferentes tamaños demuestran que el agente es capaz de aprender políticas coherentes que guían hacia el objetivo de manera eficiente. La arquitectura cognitiva de SOAR proporciona estructura e interpretabilidad al proceso de aprendizaje, mientras que el algoritmo SARSA aporta la capacidad de adaptación basada en experiencia. Particularmente interesante resultó la aplicación en el caso de la casa de Ea-Nasir, donde fue necesario introducir penalizaciones por celdas visitadas para evitar bucles y lograr la convergencia del entrenamiento.

Como trabajo futuro, sería interesante implementar estrategias de epsilon-decay para balancear de forma más dinámica la exploración y explotación durante el entrenamiento, así podría acelerar la convergencia en laberintos de mayor complejidad. Otro experimento posible (y quizás el más interesante) sería el de activar el *chunking* que el soar provee, para ver qué tan bien interactúa con el algoritmo de RL.

En cualquier caso, este proyecto resultó ser un experimento interesante que permitió explorar de primera mano la interacción entre arquitecturas cognitivas y aprendizaje por refuerzo, revelando tanto las posibilidades como los desafíos de integrar estos paradigmas para construir agentes más inteligentes y adaptativos.

A. Código Completo SOAR

A.1. Reglas de Elaboración

```
1  # Copiar recompensa al reward-link
2  sp {elaborate*reward*copy-to-reward-link
3      (state <s> ^io.input-link.reward-info <ri>
4          ^reward-link <rl>)
5      (<ri> ^value <v>)
6  -->
7      (<rl> ^reward <r>)
8      (<r> ^value <v>)
9  }
10
11 # Calcular si esta en el objetivo
12 sp {elaborate*at-goal
13     (state <s> ^io.input-link <il>)
14     (<il> ^robot <r> ^goal <g>)
15     (<r> ^x <x> ^y <y>)
16     (<g> ^x <x> ^y <y>)
```

```

17 -->
18     (<s> ^at-goal true)
19 }
20
21 # Calcular distancia al objetivo
22 sp {elaborate*distance-to-goal
23     (state <s> ^io.input-link <il>)
24     (<il> ^robot <r> ^goal <g>)
25     (<r> ^x <rx> ^y <ry>)
26     (<g> ^x <gx> ^y <gy>)
27 -->
28     (<s> ^distance-to-goal (+ (abs (- <rx> <gx>))
29                             (abs (- <ry> <gy>))))
30 }

```

A.2. Reglas de Propuesta

```

1  # Proponer movimiento arriba
2  sp {propose*move*up
3      (state <s> ^io.input-link.sensors <sens>)
4      (<sens> ^up path)
5  -->
6      (<s> ^operator <o> +)
7      (<o> ^name move ^direction up)
8  }
9
10 # Proponer movimiento abajo
11 sp {propose*move*down
12     (state <s> ^io.input-link.sensors <sens>)
13     (<sens> ^down path)
14 -->
15     (<s> ^operator <o> +)
16     (<o> ^name move ^direction down)
17 }
18
19 # Proponer movimiento izquierda
20 sp {propose*move*left
21     (state <s> ^io.input-link.sensors <sens>)
22     (<sens> ^left path)
23 -->
24     (<s> ^operator <o> +)
25     (<o> ^name move ^direction left)
26 }
27
28 # Proponer movimiento derecha
29 sp {propose*move*right
30     (state <s> ^io.input-link.sensors <sens>)
31     (<sens> ^right path)

```

```

32 -->
33     (<s> ^operator <o> +)
34     (<o> ^name move ^direction right)
35 }

```

A.3. Templates de RL

```

1  # Template RL para movimiento arriba
2  sp {rl*template*move*up
3      (state <s> ^operator <o> +
4          ^io.input-link.robot <r>)
5      (<r> ^x <x> ^y <y>)
6      (<o> ^name move ^direction up)
7  -->
8      (<s> ^operator <o> = 0)
9  }
10
11 # Template RL para movimiento abajo
12 sp {rl*template*move*down
13     (state <s> ^operator <o> +
14         ^io.input-link.robot <r>)
15     (<r> ^x <x> ^y <y>)
16     (<o> ^name move ^direction down)
17 -->
18     (<s> ^operator <o> = 0)
19 }
20
21 # Template RL para movimiento izquierda
22 sp {rl*template*move*left
23     (state <s> ^operator <o> +
24         ^io.input-link.robot <r>)
25     (<r> ^x <x> ^y <y>)
26     (<o> ^name move ^direction left)
27 -->
28     (<s> ^operator <o> = 0)
29 }
30
31 # Template RL para movimiento derecha
32 sp {rl*template*move*right
33     (state <s> ^operator <o> +
34         ^io.input-link.robot <r>)
35     (<r> ^x <x> ^y <y>)
36     (<o> ^name move ^direction right)
37 -->
38     (<s> ^operator <o> = 0)
39 }

```

A.4. Reglas de Aplicación

```
1  # Crear comando de movimiento
2  sp {apply*move*create-command
3      (state <s> ^operator <o>
4          ^io.output-link <ol>)
5      (<o> ^name move ^direction <dir>)
6  -->
7      (<ol> ^move <m>)
8      (<m> ^direction <dir>)
9  }
10
11 # Limpiar comandos completados
12 sp {apply*move*remove-completed
13     (state <s> ^io.output-link <ol>)
14     (<ol> ^move <m>)
15     (<m> ^status complete)
16 -->
17     (<ol> ^move <m> -)
18 }
```

B. Configuración JSON

B.1. Ejemplo de Estadísticas Guardadas

```
1  {
2      "summary": {
3          "total_episodes": 100,
4          "successful_episodes": 89,
5          "success_rate": 89.0,
6          "overall_avg_steps": 35.7
7      },
8      "recent_performance": {
9          "window_size": 20,
10         "successes": 18,
11         "success_rate": 90.0,
12         "avg_steps": 32.5,
13         "avg_reward": 485.3
14     },
15     "maze_info": {
16         "width": 7,
17         "height": 7,
18         "goal_position": [6, 6],
19         "spawn_positions": [[0, 0]]
20     },
21     "episode_history": [
22         {
23             "episode": 1,
```



```

24     "steps": 125,
25     "success": false,
26     "total_reward": -62.5,
27     "spawn": [0, 0],
28     "goal": [6, 6],
29     "cells_discovered": 38
30 },
31 {
32     "episode": 2,
33     "steps": 35,
34     "success": true,
35     "total_reward": 485.3,
36     "spawn": [0, 0],
37     "goal": [6, 6],
38     "cells_discovered": 25
39 }
40 ]
41 }

```

Referencias

- [1] University of Michigan SOAR Group, *SOAR Manual: Reinforcement Learning*, https://soar.eecs.umich.edu/soar_manual/05_ReinforcementLearning/, Accedido: Enero 2026.
- [2] Wikipedia contributors, *Ea-nāṣir — House Plan*, https://en.wikipedia.org/wiki/Ea-n%C4%81%E1%B9%A3ir#/media/File:Ur_1_old_street.svg, Accedido: Enero 2026.
- [3] *The Oldest Complaint Letter*, YouTube, <https://www.youtube.com/watch?v=mgiJAYS5Ye4>, Accedido: Enero 2026.